NFL Database Project
Phase 2

Nicholas Wichman
nswichma@buffalo.edu

Faiyaz Rafee
faiyazra@buffalo.edu

I. SQL Queries

In this section we will discuss the 10 SQL queries we have tested our database with and include the results. The heading of each subsection is the filename of the SQL file in our deliverables.

1. Insert Players

This is a very simple insertion query we use to insert a player, in this case one of us, into the players database. This would be used when a player joins a new team. The player inserted can be shown below with playerid 2000.

Query:

```
INSERT INTO players (playerid, playername, position, teamid, salary, yearjoined)
VALUES(2000, 'Nicholas Wichman', 'LB', 4, 100, 2026)
```

Results:

	playerid [PK] integer	playername text	position text	teamid integer	salary integer	yearjoined integer
1	209	Kevin Smith	CB	4	870524	2020
2	210	Justin Moore	CB	4	38995382	2025
3	211	Eric Anderson	QB	4	38092580	2020
4	212	Anthony Martin	WR	4	25961843	2023
5	2000	Nicholas Wichm…	LB	4	100	2026

2. Delete Players

This is a very simple deletion query that deletes a player given their playerid as it is the primary key. We will add a trigger thar results in the deletion of this players offensive and defensive statistics as well. For the example we deleted the previously added player as seen below.

Query:

```
--Simple Delete Query For Deleting Players
--Deletes Players By playerid(PK)
--This should trigger the deletion of the players stats pages
DELETE FROM players
WHERE playerid = 2000
```

Results:

	playerid [PK] integer	playername text	position text	teamid integer	salary integer	yearjoined integer
1	209	Kevin Smith	CB	4	870524	2020
2	210	Justin Moore	CB	4	38995382	2025
3	211	Eric Anderson	QB	4	38092580	2020
4	212	Anthony Mar…	WR	4	25961843	2023

3. Update Coaches Salary

This is a very simple update query that updates a coaches salary given their coached(PK). For the example we update coached = 4 to a salary of 5000 as seen below.

Query:

```
--Updates a coaches salary given their coachid(PK)
UPDATE coaches
SET salary = 5000 --Change this to any salary
WHERE coachid = 4 --Primary Key for coaches
```

Results:

	coachid [PK] integer	coachname text	teamid integer	teamname text	salary integer	yearhired integer
1	1	Jonathan Ga…	1	Arizona Cardinals	5000000	2023
2	2	Raheem Morr…	2	Atlanta Falcons	4500000	2024
3	3	John Harbau…	3	Baltimore Ravens	12000000	2008
4	4	Sean McDer…	4	Buffalo Bills	5000	2017

4. Highest Salary Per Position

This is a query for finding the highest paid player at every position. It uses teams JOIN players in order to display the teamname, salary, playername and position for each of the players. This query also uses both GROUP BY for the position and ORDER BY for the salary. The results can be seen below.

Query:

```
-- Query for finding each positons highest played player
SELECT teamname, salary, playername, position
FROM teams JOIN players
on teams.teamid = players.teamid
WHERE salary in (SELECT MAX(Salary)
    FROM teams JOIN players
    on teams.teamid = players.teamid
    GROUP by position)
ORDER BY SALARY DESC
```

Results:

	teamname text	salary integer	playername text	position text
1	New England Patrio...	44995331	Eric Garcia	K
2	Carolina Panthers	44917172	John Moore	QB
3	Jacksonville Jaguars	44872217	Brian Williams	LB
4	Atlanta Falcons	44858432	David Martin	P
5	Cleveland Browns	44856682	John Jackson	WR
6	Los Angeles Charg...	44816079	Mark Hernandez	OL
7	Miami Dolphins	44767722	David Hernand...	S
8	Minnesota Vikings	44459231	John Moore	TE
9	Chicago Bears	44325275	Kevin Johnson	DL
10	Arizona Cardinals	44155420	Eric Garcia	CB
11	Kansas City Chiefs	43793561	Kevin Garcia	RB

5. Max Salary Team

This query is for finding the highest player on every team. Much like the previous query it uses teams JOIN players and ORDER BY and GROUP BY. The results can be seen below.

Query:

```sql
-- Query for finding each teams highest played player
SELECT teamname, salary, playername, position
FROM teams JOIN players
on teams.teamid = players.teamid
WHERE salary in (SELECT MAX(Salary)
    FROM teams JOIN players
    on teams.teamid = players.teamid
    GROUP by teamname)
ORDER BY SALARY DESC
```

Results:

	teamname text	salary integer	playername text	position text
1	New England Patriots	44995331	Eric Garcia	K
2	Carolina Panthers	44917172	John Moore	QB
3	Jacksonville Jaguars	44872217	Brian Williams	LB
4	Atlanta Falcons	44858432	David Martin	P
5	Cleveland Browns	44856682	John Jackson	WR

6. Players Year Joined By Year

This query counts how many players joined their team for each year and displays the year joined along with the number of players that joined there team that year. The results can be seen below.

Query:

```sql
SELECT yearjoined, Count(*) AS count
FROM players
GROUP BY yearjoined
ORDER BY yearjoined;
```

Results:

	yearjoined integer	count bigint
1	2015	157
2	2016	152
3	2017	145
4	2018	165
5	2019	152
6	2020	139
7	2021	159
8	2022	147
9	2023	160
10	2024	148
11	2025	172

7. Top 10 Players for Each Stat

Both of these queries are very similar, so we are only counting them as one as they both find the top 10 players for a given statistic. For the current quires they find the top 10 in passyards and tackles. The results for the top 10 tackles can be seen below.

Queries:

```sql
--Returns the top 10 players in tackles
--Change tackles to any defensive stat to display top 10
SELECT playername, playerid, tackles
FROM defensivestats
ORDER BY tackles DESC
LIMIT 10
```

```sql
--Returns the top 10 players in passyards
--Change passyards to any offensive stat to display top 10
SELECT playername, playerid, passyards
FROM offensivestats
ORDER BY passyards DESC
LIMIT 10
```

Results:

	playername text	playerid integer	tackles integer
1	Andrew Tho...	1264	1729
2	Mark Moore	1099	1724
3	Daniel Garcia	955	1688
4	Justin Jacks...	595	1663
5	Joshua Wilson	217	1604
6	Brandon Bro...	1074	1563
7	David Moore	593	1554
8	James Jones	661	1529
9	Brian Thomas	1221	1467
10	Mark Jones	279	1443

8. Team Overview

This query gives a full overview of each team displaying the teamid, name, city, stadium name, coaches name, GM's name

and owners name. This is accomplished by joining 5 separate tables. The results can be seen below.

Query:

```
SELECT
    teams.teamid,
    teams.teamname,
    teams.city,
    stadiums.stadiumname,
    coaches.coachname,
    general_managers.gmname,
    owners.ownername
FROM teams
JOIN stadiums on teams.teamname = stadiums.teamname
JOIN coaches on teams.teamid = coaches.teamid
JOIN general_managers on teams.teamid = general_managers.teamid
JOIN owners on teams.teamid = owners.teamid
```

Results:

	teamid integer	teamname text	city text	stadiumname text	coachname text	gmname text	ownername text
1	1	Arizona Cardinals	Glendale	State Farm Stadium	Jonathan Ga...	Monti Ossenfort	Michael Bidwill
2	2	Atlanta Falcons	Atlanta	Mercedes-Benz Stadium	Raheem Morr...	Terry Fontenot	Arthur Blank
3	3	Baltimore Ravens	Baltimore	M&T Bank Stadium	John Harbau...	Eric DeCosta	Steve Bisciotti
4	4	Buffalo Bills	Orchard Park	Highmark Stadium	Sean McDer...	Brandon Beane	Terry Pegula

9. Insert Player Procedure

This is a simple procedure that inserts players into the players table. This is a very common use case for a query so creating a procedure allows us to not have to constantly rewrite the query. For our example we called this to insert Faiyaz with a player id of 2001 as shown below.

Procedure:

```
--Procedure to insert players
CREATE PROCEDURE InsertPlayer(
    IN p_playerid INT,
    IN p_playername TEXT,
    IN p_position TEXT,
    IN p_teamid INT,
    IN p_salary INT,
    IN p_yearjoined INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO players (playerid, playername, position, teamid, salary, yearjoined)
    VALUES (p_playerid, p_playername, p_position, p_teamid, p_salary, p_yearjoined);
END;
$$;
```

Procedure Call:

```
--Calls insert player
CALL InsertPlayer(2001, 'Faiyaz', 'RB', 4, 500, 2026);
```

Results:

	playerid [PK] integer	playername text	position text	teamid integer	salary integer	yearjoined integer
1	210	Justin Moore	CB	4	38995382	2025
2	211	Eric Anderson	QB	4	38092580	2020
3	212	Anthony Mar...	WR	4	25961843	2023
4	2001	Faiyaz	RB	4	500	2026

10. Team Payroll Summary

This query gets the payroll detail summarized by joining on teams and players. This shows name of the team, number of players, player salary totals, average salary, and highest player salary. This is done by using JOIN, GROUP BY, HAVING and ORDER BY, and HAVING, with the results of the top 10 shown below.

Query:

```
SELECT
    t.teamname,
    COUNT(p.playerid) AS playerCount,
    SUM(p.salary) AS totalPlayerSalary,
    ROUND(AVG(p.salary), 2) AS averagePlayerSalary,
    MAX(p.salary) AS highestPlayerSalary
FROM teams t
JOIN players p
    ON t.teamid = p.teamid
GROUP BY t.teamname
HAVING COUNT(p.playerid) > 0
ORDER BY totalPlayerSalary DESC;
```

Results:

	teamname text	playercount bigint	totalplayersalary bigint	averageplayersalary numeric	highestplayersalary integer
1	Los Angeles Chargers	53	1357137170	25606361.70	44816079
2	Washington Command...	53	1346560001	25406792.47	43513889
3	Arizona Cardinals	53	1341303383	25307611.00	44698714
4	New York Giants	53	1294954721	24433107.94	44686273
5	Minnesota Vikings	53	1289700000	24333962.26	44459231
6	Carolina Panthers	53	1285820225	24260758.96	44917172
7	Green Bay Packers	53	1282984890	24207262.08	42848702
8	Las Vegas Raiders	53	1270489961	23971508.70	44594547
9	Detroit Lions	53	1266161252	23889834.94	44091983
10	Cincinnati Bengals	53	1258245745	23740485.75	44475157

2. Trigger Failure Handling

We have created a function that will check if the salary of a newly added or updated player is negative, obviously the salary of one of our players cannot be negative. If the salary is negative an exception will be raised that will explain why the insertion or update was aborted. We then created a trigger that calls this function before insert or update on players. Finally, we tested this by calling our procedure InsertPlayer with a negative salary:

Code:

```sql
--Create Fucntion to check if salary < 0
CREATE OR REPLACE FUNCTION salary_check()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.salary < 0 THEN
    RAISE EXCEPTION 'Insertion Or Update Failed: Salary cannot be < 0';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

--Drop tirgger so you can rerun this query
DROP TRIGGER IF EXISTS salary_check_trigger ON players;

--Create Trigger calling salary_check()
CREATE TRIGGER salary_check_trigger
BEFORE INSERT OR UPDATE ON players
FOR EACH ROW
EXECUTE FUNCTION salary_check();

--Call our procedure with a value that would fail
CALL InsertPlayer(2002, 'Failure', 'NA', 4, -500, 2026)
```

Results:

```
ERROR:  Insertion Or Update Failed: Salary cannot be < 0
CONTEXT:  PL/pgSQL function salary_check() line 4 at RAISE
SQL statement "INSERT INTO players (playerid, playername, position, teamid, salary, yearjoined)
       VALUES (p_playerid, p_playername, p_position, p_teamid, p_salary, p_yearjoined)"
PL/pgSQL function insertplayer(integer,text,text,integer,integer,integer) line 3 at SQL statement

SQL state: P0001
```

3. Indexing & Analysis

In this section we will cover three problematic queries in our database. We will discuss the initial cost of these queries, implemented an index and finally show the improved cost using indexing.

3.1 Ordering Players by Salary

The query shown here is selecting all attributes from players and ordering by their salary decreasing. Right now we have a little over 1600 players in our database, sorting this everytime we need to order by salary will lead to long execution times. To solve this issue creating an index will allow accessing salary values in sorted order faster and will lower overhead. Originally using EXPLAIN ANALYZE on this query revealed that it used a quicksort algorithm to sort the salary key, this led to a planning time of 3.012 ms and an execution time of 5.151ms. Using a simple CREATE INDEX on players (salary DESC) led to the query having a planning time of 2.483 ms and an execution time of .949 ms. The EXPLAIN ANALZYE output for before and after the indexing is shown below:

Query:

Query | Query History

```sql
SELECT *
FROM players
ORDER BY salary DESC
```

Before Indexing:

	QUERY PLAN text
1	Sort (cost=121.93..126.17 rows=1696 width=31) (actual time=2.524..2.672 rows=1696.00 loops=1)
2	Sort Key: salary DESC
3	Sort Method: quicksort Memory: 135kB
4	Buffers: shared hit=17 dirtied=1
5	-> Seq Scan on players (cost=0.00..30.96 rows=1696 width=31) (actual time=0.303..1.186 rows=1696.00 loo...
6	Buffers: shared hit=14 dirtied=1
7	Planning:
8	Buffers: shared hit=42
9	Planning Time: 3.012 ms
10	Execution Time: 5.151 ms

After indexing:

	QUERY PLAN text
1	Index Scan using index_slaary on players (cost=0.28..109.69 rows=1696 width=31) (actual time=0.048..0.564 rows=1696.00 loo...
2	Index Searches: 1
3	Buffers: shared hit=1575 read=6
4	Planning:
5	Buffers: shared hit=53 read=1
6	Planning Time: 2.483 ms
7	Execution Time: 0.949 ms

3.2 Ordering Players by Last Names

The query shown here displays the playerid, playername and teamid from the players table order by last name. Since our players table has over 1600 records sorting the table in any regard will lead to a high cost for both memory overhead and execution time. With players constantly being added the performance of this query will get worse over time. We will improve the performance by creating a index on the playername going reverse alphabetically. Before creating our index this query has a planning time of .920 ms and execution time of 4.858 ms. After indexing we have a planning time of .122 ms and execution time of .685 ms. The query, index and EXPLAIN ANALYZE are shown below:

Query:

Query | Query History

```sql
EXPLAIN ANALYZE
SELECT
    playerid,
    playername,
    teamid
FROM players
ORDER BY playername DESC
```

Index:

Query | Query History

```sql
CREATE INDEX player_name_index
ON players (playername DESC)
```

Before Indexing:

	QUERY PLAN text
1	Sort (cost=121.93..126.17 rows=1696 width=31) (actual time=3.314..3.368 rows=1696.00 loops=1)
2	Sort Key: playername DESC
3	Sort Method: quicksort Memory: 135kB
4	Buffers: shared hit=14
5	-> Seq Scan on players (cost=0.00..30.96 rows=1696 width=31) (actual time=0.045..0.214 rows=1696.00 loo…
6	Buffers: shared hit=14
7	Planning:
8	Buffers: shared hit=6 dirtied=1
9	Planning Time: 0.920 ms
10	Execution Time: 4.858 ms

After Indexing:

	QUERY PLAN text
1	Index Scan using player_name_index on players (cost=0.28..105.72 rows=1696 width=21) (actual time=0.025..0.615 rows=1696.00 loo…
2	Index Searches: 1
3	Buffers: shared hit=1452
4	Planning Time: 0.122 ms
5	Execution Time: 0.685 ms

3.3 Repeated stat lookup for Players

The query shown below joins players with teams and retrieves offensive and defensive stats with repeated and related subqueries, which makes this awfully inefficient at first since it looks up passing yards, passing touchdowns, tackles and sacks by matching the playerID to the offensive and defensive records. Without indexing, this has a very high cost as it needs to repeatedly look up the same player id in the same tables, making it a costly operation, and has a planning time of 0.49 ms and execution time of 415ms. After indexing we have a planning time of 1ms and execution time of 19ms, dropping total execution time tremendously. With players getting added this difference will only grow larger as well.

Query:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    t.teamname,
    p.playerid,
    p.playername,
    p.salary,
    (SELECT o.passyards
     FROM offensivestats o
     WHERE o.playerid = p.playerid) AS passyards,
    (SELECT o.passtds
     FROM offensivestats o
     WHERE o.playerid = p.playerid) AS passtds,
    (SELECT d.tackles
     FROM defensivestats d
     WHERE d.playerid = p.playerid) AS tackles,
    (SELECT d.sacks
     FROM defensivestats d
     WHERE d.playerid = p.playerid) AS sacks
FROM players p
JOIN teams t
    ON p.teamid = t.teamid
ORDER BY passyards DESC, passtds DESC, tackles DESC, sacks DESC
LIMIT 25;
```

Index:

```
CREATE INDEX idx_offensivestats_playerid
ON offensivestats (playerid);
```

Before Indexing:

	QUERY PLAN text
1	Limit (cost=252450.38..252450.44 rows=25 width=54) (actual time=415.025..415.029 rows=25.00 loops=1)
2	Buffers: shared hit=108559
3	-> Sort (cost=252450.38..252454.62 rows=1696 width=54) (actual time=415.024..415.025 rows=25.00 loops=1)
4	Sort Key: ((SubPlan 1)) DESC, ((SubPlan 2)) DESC, ((SubPlan 3)) DESC, ((SubPlan 4)) DESC
5	Sort Method: top-N heapsort Memory: 29kB
6	Buffers: shared hit=108559
7	-> Hash Join (cost=1.72..252402.52 rows=1696 width=54) (actual time=1.658..414.537 rows=1696.00 loops=1)
8	Hash Cond: (p.teamid = t.teamid)
9	Buffers: shared hit=108559
10	-> Seq Scan on players p (cost=0.00..30.96 rows=1696 width=25) (actual time=0.009..0.138 rows=1696.00 loops=1)
11	Buffers: shared hit=14
12	-> Hash (cost=1.32..1.32 rows=32 width=21) (actual time=0.029..0.029 rows=32.00 loops=1)
13	Buckets: 1024 Batches: 1 Memory Usage: 10kB
14	Buffers: shared hit=1
15	-> Seq Scan on teams t (cost=0.00..1.32 rows=32 width=21) (actual time=0.004..0.009 rows=32.00 loops=1)
16	Buffers: shared hit=1
17	SubPlan 1
18	-> Seq Scan on offensivestats o (cost=0.00..37.20 rows=1 width=4) (actual time=0.028..0.058 rows=1.00 loops=16…
19	Filter: (playerid = p.playerid)
20	Rows Removed by Filter: 1695
21	Buffers: shared hit=27136
22	SubPlan 2
23	-> Seq Scan on offensivestats o_1 (cost=0.00..37.20 rows=1 width=4) (actual time=0.031..0.060 rows=1.00 loops=…
24	Filter: (playerid = p.playerid)
25	Rows Removed by Filter: 1695
26	Buffers: shared hit=27136
27	SubPlan 3
28	-> Seq Scan on defensivestats d (cost=0.00..37.20 rows=1 width=4) (actual time=0.026..0.062 rows=1.00 loops=16…
29	Filter: (playerid = p.playerid)
30	Rows Removed by Filter: 1695
31	Buffers: shared hit=27136
32	SubPlan 4
33	-> Seq Scan on defensivestats d_1 (cost=0.00..37.20 rows=1 width=4) (actual time=0.026..0.060 rows=1.00 loops=…
34	Filter: (playerid = p.playerid)
35	Rows Removed by Filter: 1695
36	Buffers: shared hit=27136
37	Planning:
38	Buffers: shared hit=40
39	Planning Time: 0.489 ms
40	Execution Time: 415.103 ms

After Indexing:

	QUERY PLAN text
1	Limit (cost=56358.86..56358.92 rows=25 width=54) (actual time=18.970..18.981 rows=25.00 loops=1)
2	Buffers: shared hit=20355 read=12
3	-> Sort (cost=56358.86..56363.10 rows=1696 width=54) (actual time=18.968..18.976 rows=25.00 loops=1)
4	Sort Key: ((SubPlan 1)) DESC, ((SubPlan 2)) DESC, ((SubPlan 3)) DESC, ((SubPlan 4)) DESC
5	Sort Method: top-N heapsort Memory: 29kB
6	Buffers: shared hit=20355 read=12
7	-> Hash Join (cost=1.72..56311.00 rows=1696 width=54) (actual time=0.155..17.855 rows=1696.00 loops=1)
8	Hash Cond: (p.teamid = t.teamid)
9	Buffers: shared hit=20355 read=12
10	-> Seq Scan on players p (cost=0.00..30.96 rows=1696 width=25) (actual time=0.018..0.330 rows=1696.00 loops=1)
11	Buffers: shared hit=14
12	-> Hash (cost=1.32..1.32 rows=32 width=21) (actual time=0.029..0.030 rows=32.00 loops=1)
13	Buckets: 1024 Batches: 1 Memory Usage: 10kB
14	Buffers: shared hit=1
15	-> Seq Scan on teams t (cost=0.00..1.32 rows=32 width=21) (actual time=0.005..0.011 rows=32.00 loops=1)
16	Buffers: shared hit=1
17	SubPlan 1
18	-> Index Scan using idx_offensivestats_playerid on offensivestats o (cost=0.28..8.29 rows=1 width=4) (actual time=0.002..0.002 rows=1.00 loops=16...
19	Index Cond: (playerid = p.playerid)
20	Index Searches: 1696
21	Buffers: shared hit=5082 read=6
22	SubPlan 2
23	-> Index Scan using idx_offensivestats_playerid on offensivestats o_1 (cost=0.28..8.29 rows=1 width=4) (actual time=0.002..0.002 rows=1.00 loops=...
24	Index Cond: (playerid = p.playerid)
25	Index Searches: 1696
26	Buffers: shared hit=5088
27	SubPlan 3
28	-> Index Scan using idx_defensivestats_playerid on defensivestats d (cost=0.28..8.29 rows=1 width=4) (actual time=0.002..0.002 rows=1.00 loops=1...
29	Index Cond: (playerid = p.playerid)
30	Index Searches: 1696
31	Buffers: shared hit=5082 read=6
32	SubPlan 4
33	-> Index Scan using idx_defensivestats_playerid on defensivestats d_1 (cost=0.28..8.29 rows=1 width=4) (actual time=0.002..0.002 rows=1.00 loops...
34	Index Cond: (playerid = p.playerid)
35	Index Searches: 1696
36	Buffers: shared hit=5088
37	Planning:
38	Buffers: shared hit=54 read=2
39	Planning Time: 1.053 ms
40	Execution Time: 19.095 ms

4. Individual Contributions

We divided the work up fairly and evenly for this project. We decided that Nicholas Wichman should do the data generation and then both members should create the database together on one machine. As for the query section Nicholas Wichman did the first 5 mentioned in our report whereas Faiyaz Rafee did the other 5 mentioned. We worked on the trigger together after finishing the queries. Finally, the first 2 problematic queries were made by Nicholas Wichman and the other 1 was made by Faiyaz Rafee. Overall, we feel as if the work was dived fairly and evenly amongst teammates.